

1] ■ Denial of Service (DoS / DDoS) & Excessive Usage

■ 1. INTRODUCTION

Denial of Service (DoS) and Distributed Denial of Service (DDoS) attacks are among the most common and dangerous cybersecurity threats. Their primary goal is **not to steal data**, but to **make a system, service, or network unavailable to legitimate users**.

In modern cloud-based systems, these attacks can also cause **massive financial loss** due to resource consumption, making them both a **technical and economic threat**.

■ 2. WHAT (DETAILED EXPLANATION)

■ DoS (Denial of Service)

A **DoS attack** occurs when a **single attacker or machine** sends a large number of requests to a server, overwhelming it.

Key Idea:

■ One source → Many requests → Server crash

A script sends 10,000 requests per second to a login API.

■ DDoS (Distributed Denial of Service)

A **DDoS attack** is an advanced version of DoS where **multiple machines (often thousands)** attack a system simultaneously.

These machines are usually:

- * Infected computers
- * IoT devices (CCTV, routers)
- * Controlled by attacker → called **Botnet**

Key Idea:

■ Many sources → Massive traffic → Complete service failure

■ Excessive Usage (Important Concept)

This is often ignored but very important.

It refers to:

- * **Overuse of system resources**
- * Can be **intentional or unintentional**
- * Exploits **poor system design**
- * Running expensive DB queries repeatedly
- * Uploading large files continuously
- * Calling APIs without limits

■ Unlike DDoS, this may come from **valid users or APIs**

■■ 3. TYPES OF DoS/DDoS ATTACKS (DETAILED)

■ 1. Volumetric Attacks

What it does:

Floods the network with massive traffic to ****consume bandwidth****

- * UDP Flood
- * ICMP Flood (Ping flood)

Working:

- * Sends millions of packets per second
- * Network gets saturated
- * Legit users cannot access service

If server supports 1 Gbps → attacker sends 50 Gbps

■ Result: Network collapse

■ 2. Protocol Attacks (SYN Flood)

What it targets:

Weakness in network protocols (TCP handshake)

Normal TCP Process:

1. Client → SYN
2. Server → SYN-ACK
3. Client → ACK

Attack:

- * Attacker sends SYN
- * Server replies SYN-ACK
- * Attacker NEVER sends ACK

■ Connection stays half-open

Result:

- * Server memory fills
- * New connections rejected

■ 3. Application Layer Attacks (Most Dangerous)

What it targets:

Specific endpoints like:

- * `/login``
- * `/search``
- * `/checkout``

Why dangerous:

- * Looks like real user traffic
- * Hard to detect

Sending thousands of login requests per second

■ Server CPU gets overloaded

■ 4. Excessive Usage Attacks

What happens:

- * Instead of flooding traffic, attacker uses ****heavy operations repeatedly****

- * Complex SQL queries
- * Image processing APIs
- * Report generation APIs

■ These consume high CPU → slow system

■ 4. WHY ATTACKERS DO THIS (DETAILED)

■ 1. Financial Damage

- * Downtime = loss of customers
- * E-commerce sites lose revenue per minute

■ 2. Competitive Sabotage

- * Attack competitor during:

- * Sales
- * Product launch

■ 3. Hacktivism

- * Political or social protest

Attack government websites

■ 4. Extortion (Ransom DDoS)

- * "Pay us or we will keep your site down"

■■ 5. Cloud Cost Attack

- * Force server to auto-scale
- * Increase cloud billing

■ 6. Reputation Damage

- * Users lose trust in system reliability

■■ 5. HOW THE ATTACK WORKS (STEP-BY-STEP)

Step 1: Build Botnet

- * Infect thousands of devices using malware

Step 2: Command & Control (C&C;)

- * Attacker controls all bots remotely

Step 3: Launch Attack

- * All bots send requests simultaneously

Step 4: Resource Exhaustion

- * CPU usage ↑
- * Memory usage ↑
- * Bandwidth exhausted

Step 5: Service Crash

- * Legit users cannot access system

■■ 6. HOW TO PREVENT (VERY DETAILED)

■ 1. Rate Limiting (MOST IMPORTANT)

Limit number of requests per:

- * IP
 - * User
 - * API key
-
- * Max 100 requests/min per user

■ Prevents abuse early

■ 2. Throttling

Slows down excessive users instead of blocking immediately

■ 3. CDN (Content Delivery Network)

- * Cloudflare
- * Akamai

Benefits:

- * Distributes traffic globally
- * Absorbs large traffic spikes

■ 4. Load Balancing

- * Distributes traffic across multiple servers

■ No single server gets overloaded

■ 5. Anycast Routing

- * Same IP → multiple data centers

■ Traffic automatically routed to nearest healthy server

■ 6. Web Application Firewall (WAF)

- * Filters malicious traffic
- * Blocks suspicious patterns

■ 7. API Gateway Protection

- * Set quotas
- * Limit heavy operations

■ 8. Caching

- * Reduce server load
- * Cache search results

■ 9. Auto Scaling (Cloud)

- * Automatically increase servers during traffic spikes

■ 10. Traffic Monitoring

- * Detect unusual spikes
- * Alert admins

■ 11. Blacklisting & Whitelisting

- * Block malicious IPs
- * Allow trusted users

■ 12. Circuit Breaker Pattern

- * Stop processing when system overloaded

■ 13. CAPTCHA

- * Prevent bots

■ GitHub DDoS Attack (2018)

- * One of the largest attacks (~1.3 Tbps)
- * Used Memcached servers

■ Dyn DNS Attack

- * Affected:

- * Twitter
- * Netflix
- * Reddit

■ AWS Shield

- * Amazon service to protect against DDoS

■ Node.js Rate Limiting

```
```js
const rateLimit = require("express-rate-limit");

const limiter = rateLimit({
 windowMs: 1 * 60 * 1000,
 max: 100
});

app.use(limiter);
```
```

```
```csharp
builder.Services.AddRateLimiter(options =>
{
 options.AddFixedWindowLimiter("fixed", opt =>
 {
 opt.PermitLimit = 100;
 opt.Window = TimeSpan.FromMinutes(1);
 });
});
```
```

- * DoS = **single attacker**
- * DDoS = **multiple systems (botnet)**
- * Application layer attacks are **hardest to detect**
- * Rate limiting = **first line of defense**
- * Excessive usage = **design flaw exploitation**
- * CDN is **best real-world solution**

- DoS/DDoS = **Overload system → crash service**
- Excessive Usage = **Misuse resources → slow system**
- Prevention = **Rate limit + CDN + monitoring**

If this is the level you want, I can now:

- * Do the same **3-page deep explanation** for:

- * Abuse of Functionality
- * Access Violation
- * Exploits
- * Then combine everything into a **perfect final PDF (topper-level)**

Just say: **“next topic”** ■

■ 2. Abuse of Functionality (Beginner + Deep Explanation)

■ 1. SIMPLE INTRODUCTION (VERY EASY)

Imagine you have a system (like a website or app).

- The system is working perfectly.
- There is NO bug.
- But someone ****uses it in a wrong way**** to get benefit.

That is called:

> ****Abuse of Functionality = Using a system correctly, but with bad intention****

- * Website allows ****buy 1 ticket per user****
- * A person creates ****100 fake accounts****
- * Buys all tickets

- System worked correctly
- But it was ****abused****

■ WHY THIS IS DANGEROUS

- * Hard to detect (looks like real users)
- * No hacking needed
- * Can cause:

- * Money loss
- * Data theft
- * Unfair advantage

■■ 2. TYPES OF ABUSE (SUPER CLEAR + DETAILED)

■ 1. SCALPING (Very Easy Understanding)

■ WHAT (Simple)

Scalping means:

> Buying things very fast using bots and selling them at higher price

- * Concert tickets open at 10 AM
- * Within 2 seconds → SOLD OUT
- * Same tickets appear on resale at 5x price

- Humans couldn't buy
- Bots bought everything

■■ HOW IT HAPPENS (Step-by-step)

1. Attacker creates a ****bot (automatic program)****
2. Bot keeps refreshing website
3. As soon as item is available:

- * Adds to cart
- * Completes payment instantly
- 4. Repeats using many accounts

■ WHY IT'S BAD

- * Normal users suffer
- * Creates black market
- * Company reputation goes down

■■ HOW TO PREVENT (Easy + Clear)

- * CAPTCHA (Are you human?)
- * Limit 1 purchase per user
- * Queue system (like waiting line)
- * Detect bots (very fast actions)

■ 2. SCRAPING (Super Easy)

■ WHAT

Scraping means:

> Automatically collecting data from a website

- * You open Amazon → see product prices
- * Competitor creates bot → copies ALL prices

■ Now competitor always sells cheaper

■■ HOW IT WORKS

1. Bot opens website automatically
2. Reads data (price, product, users)
3. Saves data
4. Repeats thousands of times

■ WHY IT'S BAD

- * Your data is stolen
- * Server becomes slow
- * Business loss

■■ HOW TO PREVENT

- * Limit number of requests
- * Ask login (authentication)
- * CAPTCHA
- * Detect bots

■ 3. CLICK FRAUD (Very Simple)

■ WHAT

Click fraud means:

> Clicking ads again and again using bots

- * Company runs ads on Google
- * Attacker clicks ads 1000 times

■ Company pays money for fake clicks

■■ HOW IT WORKS

1. Bot visits website
2. Clicks ads automatically
3. Repeats again and again

■ WHY IT'S BAD

- * Money loss
- * Fake traffic
- * Wrong analytics

■■ HOW TO PREVENT

- * Track IP addresses
- * Detect unusual clicks
- * Block repeated users
- * Use AI tools for fraud detection

■ 4. BRUTE FORCE (VERY IMPORTANT)

■ WHAT

Brute force means:

> Trying many passwords until correct one is found

- * Password = `123456`
- * Attacker tries:

- * 123
- * 1234
- * 12345
- * 123456 ■

■■ TYPES (Simple)

1. Basic Brute Force

Try all passwords

2. Dictionary Attack

Use common passwords list:

- * password123
- * admin123

3. Credential Stuffing

Use leaked passwords from other websites

■ WHY IT'S BAD

- * Account hacking
- * Data theft
- * Money theft

■■ HOW TO PREVENT

- * Lock account after 5 tries
- * Add delay after each attempt
- * Use OTP (MFA)
- * Strong passwords

■ SIMPLE CODE IDEA

```
```python
if attempts > 5:
 block_user()
```
```

■ 3. WHY PEOPLE DO THIS (VERY EASY)

■ 1. Money

- * Sell tickets at high price
- * Earn from ad clicks

■ 2. Business Advantage

- * Copy competitor data

■ 3. Hack Accounts

- * Brute force attacks

■ 4. Cheat System

- * Use coupons multiple times
- * Fake referrals

■■ 4. HOW ATTACKERS DO IT (EASY FLOW)

1. Find feature (login, buy, search)
2. Write bot/script
3. Use many IPs
4. Send fast repeated requests
5. Avoid detection

■■ 5. BEST DEFENSE (BEGINNER FRIENDLY)

■ 1. CAPTCHA

Stops bots

■ 2. Rate Limiting

Limits requests

■ 3. MFA (OTP)

Extra security

■ 4. Monitoring

Detect unusual activity

■ 5. Behavioral Detection

Detect:

- * Very fast clicks
- * No mouse movement

■ 6. REAL-LIFE CASES (EASY)

■■ Ticket Scalping

Bots buy tickets → resell expensive

■ Amazon Scraping

Competitors copy product prices

■ Ad Fraud

Bots click ads → company loses money

■ Password Attacks

Hack accounts using password lists

* Abuse ≠ hacking

■ It is ****misuse of system****

* No bug needed

■ System works fine

* Bots are main tool

* Hard to detect

* CAPTCHA alone is not enough

■ Abuse of Functionality =

****"Using a system the right way, but for the wrong purpose"****

| Type | Simple Meaning |

| ----- | ----- |

| Scalping | Buy fast, sell expensive |

| Scraping | Copy data automatically |

| Click Fraud | Fake ad clicks |

| Brute Force | Guess passwords |

If you like this style (beginner + deep), I'll do same for:

* ****Access Violation****

* ****Exploits****

Just say: ****"next topic"**** ■

3]

■ 3. Access Violation (Beginner + Deep Explanation)

■ 1. SIMPLE INTRODUCTION

Think of your ****bank account, WhatsApp, or email****.

■ Only ***you*** should access it, right?

Now imagine:

* Someone reads your messages

* Uses your account

* Sees your password

■ That is called:

> ****Access Violation = Someone getting access to your data without permission****

You log in to your Instagram → go to a café → use public WiFi

A hacker:

- * Watches your internet activity
- * Steals your login session

■ Now he is inside your account without password

■ WHY THIS IS VERY DANGEROUS

- * Personal data stolen
- * Bank fraud
- * Identity theft
- * Privacy violation

■■ 2. TYPES OF ACCESS VIOLATION (VERY SIMPLE + DETAILED)

■ 1. SESSION HACKING (Session Hijacking)

■ WHAT (Very Simple)

When you log into a website, it gives you a **session ID** (like a temporary pass).

■ Session hacking means:

> Attacker steals this “pass” and becomes you

- * You log into Facebook
 - * Website remembers you (no login again)
- That's session

Now attacker steals that session →

■ He becomes logged-in as you

■■ HOW IT HAPPENS

- * Through unsafe WiFi
- * Through XSS attack
- * Through stealing cookies

■ WHY IT'S BAD

- * No password needed
- * Full account control
- * Very hard to detect

■■ HOW TO PREVENT

- * Use HTTPS (secure connection)
- * Logout after use
- * Use secure cookies
- * Avoid public WiFi

■ 2. SNOOPING

■ WHAT

Snooping means:

> Secretly watching or listening to your data

- * You send message on public WiFi
- * Hacker reads your data

■ Like someone reading your letters

■■ HOW IT WORKS

- * Data travels through network
- * If not encrypted → readable
- * Attacker uses tools to capture data

■ WHY IT'S BAD

- * Passwords stolen
- * Messages leaked
- * Privacy gone

■■ HOW TO PREVENT

- * Always use HTTPS websites
- * Avoid public WiFi
- * Use VPN

■ 3. MAN-IN-THE-MIDDLE (MITM)

■ WHAT

This is more dangerous.

> Attacker sits between you and the website

You → (attacker) → Website

You think:

■ You are talking to website

Reality:

■ You are talking to attacker

■■ WHAT ATTACKER CAN DO

- * Read your data
- * Change your data
- * Steal passwords

- * Fake WiFi in café
- * You connect

* Hacker controls network

■ WHY IT'S BAD

- * Invisible attack
- * Full control of communication

■■ HOW TO PREVENT

- * Use HTTPS
- * Don't connect to unknown WiFi
- * Use VPN
- * Check website certificate (lock icon ■)

■ 4. CREDENTIAL STUFFING

■ WHAT

People reuse passwords.

■ Credential stuffing means:

> Using leaked passwords from one site to login into another

- * Your password leaked from Facebook
- * You use same password in Gmail
- Attacker tries → login successful

■■ HOW IT WORKS

1. Attacker gets leaked database
2. Uses bot
3. Tries same credentials on multiple sites

■ WHY IT'S BAD

- * Very common
- * Works easily due to password reuse

■■ HOW TO PREVENT

- * Use different passwords
- * Use password manager
- * Enable MFA (OTP login)

■ EXTRA ACCESS ATTACKS (IMPORTANT)

■ Privilege Escalation

- * Normal user becomes admin

■ Token/Cookie Theft

- * Stealing login session

■ Unauthorized API Access

- * Access data without permission

■ 3. WHY ATTACKERS DO THIS (VERY SIMPLE)

■ Money

- * Bank fraud
- * Online payments

■ Data Theft

- * Personal info
- * Company secrets

■■ Spy / Surveillance

- * Monitor users

■ Identity Theft

- * Use your account as you

■■ 4. HOW ATTACK HAPPENS (STEP-BY-STEP)

1. User connects to unsafe network
2. Data is intercepted
3. Attacker steals session/password
4. Uses it to login
5. Gains access

■■ 5. HOW TO PREVENT (VERY IMPORTANT)

■ 1. HTTPS (MOST IMPORTANT)

- * Encrypts data
- * Prevents snooping

■ 2. MFA (OTP)

* Even if password stolen → safe

■ 3. Secure Cookies

* httpOnly → no JS access

* secure → only HTTPS

■ 4. Logout & Session Expiry

* Prevent long sessions

■ 5. Avoid Public WiFi

* Major attack source

■ 6. Use Strong Passwords

* Unique passwords

■ 7. Monitor Activity

* Detect unusual login

■ Public WiFi Attack

* Hacker steals login details

■ Data Breaches

* Millions of passwords leaked

■■■ MITM Attacks

* Fake WiFi hotspots

* Access violation = **Unauthorized access**

* Session hijacking = **stealing login session**

* Snooping = **watching data**

* MITM = **middle attacker**

* Credential stuffing = **reuse of leaked passwords**

* HTTPS = **most important defense**

* MFA = **strongest protection**

■ Access Violation =

"Someone gets access to your account/data without permission"

| Type | Simple Meaning |
|---------------------|-------------------------|
| Session Hacking | Steal login session |
| Snooping | Read your data |
| MITM | Intercept communication |
| Credential Stuffing | Use leaked passwords |

Now only one left:

■ **Exploits (SQL Injection, XSS, Buffer Overflow)**

Then I'll combine ALL into a **perfect final PDF (topper-level)**

Just say: **"last topic"** ■

4]

■ 4. Exploits (Beginner + Deep Explanation)

■ 1. SIMPLE INTRODUCTION

Imagine a **door with a small hidden crack**.

- * Normal users use the door properly
- * But a smart attacker finds the crack and enters

■ That crack = **vulnerability (weakness)**

■ Using it = **Exploit**

■ SIMPLE DEFINITION

> **Exploit = Using a mistake/weakness in a system to attack it**

- * Website asks for username & password
- * Attacker enters something unusual → system breaks → gives access

■ That is an exploit

■ WHY EXPLOITS ARE VERY DANGEROUS

- * Can give **full system control**
- * Can steal **entire database**
- * Can run **malicious code**
- * Often invisible

■■ 2. TYPES OF EXPLOITS (VERY SIMPLE + DETAILED)

■ 1. SQL INJECTION (SQLi)

■ WHAT (Very Simple)

Websites store data in a **database** (like a digital Excel sheet).

SQL is the language used to talk to database.

■ SQL Injection means:

> Attacker tricks the database by entering special input

Login form:

Username: admin

Password: 1234

Normal system checks:

■ Is username & password correct?

■ ATTACK INPUT

Attacker enters:

' OR 1=1 --

■ Meaning:
"Always true"

■■ WHAT HAPPENS

- * System gets confused
- * Login check bypassed
- * Attacker logs in without password

■ WHY IT'S BAD

- * Access all users data
- * Delete database
- * Change data

■ REAL-LIFE

Many websites were hacked using SQL Injection

■■ HOW TO PREVENT (Simple)

- * Never trust user input
- * Use **safe queries (parameterized queries)**
- * Validate input

■ SIMPLE IDEA

■ Don't mix user input with database commands

■ 2. CROSS-SITE SCRIPTING (XSS)

■ WHAT (Very Simple)

XSS means:

> Attacker injects **JavaScript code** into a website

And that code runs in **other users' browsers**

Website allows comments:

...

Nice product!

...

Attacker enters:

...

alert("Hacked")

...

■■ WHAT HAPPENS

- * Website stores this
- * Other users open page
- * Script runs automatically

■ Now attacker can:

- * Steal cookies
- * Steal session
- * Redirect users

■ WHY IT'S BAD

- * Affects many users
- * Steals login info
- * Can control user session

■ REAL-LIFE

- * Social media comment attacks
- * Fake popups stealing passwords

■■ HOW TO PREVENT

- * Clean user input
- * Do not allow scripts
- * Use secure output methods

■ SIMPLE SAFE IDEA

Instead of:

...

show user input directly

...

■ Show it as plain text (no execution)

■ 3. BUFFER OVERFLOW

■ WHAT (Very Simple)

Computer memory has **limited space** (like a container)

■ Buffer overflow means:

> Putting too much data into small space

Imagine:

- * Box can hold 10 letters
- * You put 50 letters

■ Extra letters spill out and damage nearby things

■■ WHAT HAPPENS

- * Memory gets corrupted
- * Attacker overwrites important data
- * Can run their own code

■ WHY IT'S VERY DANGEROUS

- * Can give **full control of system**
- * Used in serious attacks
- * Hard to detect

■ REAL-LIFE

- * Old software vulnerabilities
- * System-level hacking

■■ HOW TO PREVENT

- * Limit input size
- * Use safe programming languages (Java, Python)
- * Validate input

■ CSRF (Cross-Site Request Forgery)

■ Tricks user to perform actions without knowing

* Clicking malicious link → transfers money

■ RCE (Remote Code Execution)

■ Attacker runs code on server

■ Zero-Day Attack

■ New vulnerability with no fix yet

■ 3. WHY ATTACKERS USE EXPLOITS

■ Money

- * Steal data
- * Financial fraud

■ Control

- * Take over system

■ Data Theft

- * User info
- * Company secrets

■ Damage

- * Delete or modify data

■■ 4. HOW ATTACK WORKS (SIMPLE FLOW)

1. Find weakness in system
2. Create malicious input
3. Send to system
4. System executes it
5. Attacker gains access

■■ 5. HOW TO PREVENT (VERY IMPORTANT)

■ Input Validation

- * Check all user inputs

■ Output Encoding

* Show data safely

■ Secure Coding

* Avoid unsafe methods

■ Regular Updates

* Fix vulnerabilities

■ Security Testing

* Find bugs early

■■ SQL Injection

* Database leaks

■ XSS

* Session stealing

■ Buffer Overflow

* System hacking

* Exploit = **using weakness**

* SQL Injection = **database attack**

* XSS = **runs in browser**

* Buffer Overflow = **memory attack**

* Input validation = **most important defense**

* Parameterized queries = **best for SQLi**

■ Exploit =

Using a system mistake to attack it

| Type | Simple Meaning |

| ----- | ----- |

| SQL Injection | Trick database |

| XSS | Inject script |

| Buffer Overflow | Overflow memory |

Now you have ALL 4 topics in:

* Beginner-friendly

* Detailed

■ Say this next:

"create final premium PDF"

I'll combine EVERYTHING into:

- * Clean formatting
- * Headings + spacing
- * Tables

■